2024-SC2-COS10033-Advanced Programming

Assignment 2 Report

Name: Banuka Jayathilaka **Student ID: 105200902**

Lab class: Wednesday **Teacher: MS LUSHAKA NISANSALA**

Due Date: 14 Sep at 23:59

Date Submitted: 11 Sep at 11.15

StageD.java

```
package com.assignment2.stageD;

import java.io.File;
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;
import java.text.ParseException;
import java.text.SimpleDateFormat;
import java.util.ArrayList;
import java.util.Date;
import java.util.InputMismatchException;
import java.util.Scanner;

import com.finalTest.codes.CardPayment;
import com.finalTest.codes.ChequePayment;
import com.finalTest.codes.Payment;

public class StageD {

	// ANSI escape codes for colored console output
	public static final String ANSI_RESET = "\u001B[0m";// Reset color
	public static final String ANSI_RED = "\u001B[31m";// Red text
	public static final String ANSI_GREEN = "\u001B[32m";// Green text
	public static final String BRIGHT_BLUE = "\u001B[96m";// Bright blue text
	public static final String PURPLE = "\u001B[35m";// Purple text

	// Variable to keep track of the total sum of room bookings
	static double totalSum = 0;

	@SuppressWarnings("resource")
	public static void main(String[] args) throws IOException,
InputMismatchException {
		// Create a Scanner object for reading user input
		Scanner main = new Scanner(System.in); // Scanner for user input
		boolean except = false; // flag used to see the program is running
for the first time

		// List to hold room objects
		ArrayList<Room> Accomodation = new ArrayList<>(); // List to store
room objects

		// Declaration of Room objects
		Standard roomStandard;
		Deluxe roomDeluxe;
		Premium roomPremium;
		SpecialService_DoubleRoom roomSpecialDouble;
		SpecialService_TripleRoom roomSpecialTriple;

		// try block happens only if a room file is existing
		try {
			Scanner sc = new Scanner(new File("Rooms.txt")); // Reading
room data from file
			int x = 0; // count varaiable

			double total = sc.nextDouble(); // reading the total sum
			totalSum = total;
			int num = sc.nextInt(); // reading the number of rooms stored
```

```
                    int totalRooms = num;
                    sc.nextLine();
                    while (x < totalRooms) {
                          int tempRoomNum = sc.nextInt();
                          int tempNumOfDays = sc.nextInt();
                          sc.nextLine();
                          String line = sc.nextLine();
                          String[] tokens = line.split("/"); // room info
separated by a /
                          String[] findGuestID = line.split("#"); // special
rooms have an extra attribute - guestID
                          String tempRoomName = tokens[0];
                          String tempId = tokens[1];

                          // extracting rooms from the text file and restoring
them back to the arraylist
                          switch (tempRoomName.charAt(7)) { // info stored i such
a way the 7th character int he room name will

      // determine it's type.
                          case 'S':
                                roomStandard = new Standard(tempRoomNum,
tempRoomName, tempId, tempNumOfDays);

                                Accomodation.add(roomStandard);
                                break;
                          case 'D':
                                roomDeluxe = new Deluxe(tempRoomNum,
tempRoomName, tempId, tempNumOfDays);

                                Accomodation.add(roomDeluxe);
                                break;
                          case 'P':
                                roomPremium = new Premium(tempRoomNum,
tempRoomName, tempId, tempNumOfDays);

                                Accomodation.add(roomPremium);
                                break;
                          }

                          if (tempRoomName.charAt(8) == 'D') {
                                if (findGuestID.length == 1)
                                      roomSpecialDouble = new
SpecialService_DoubleRoom(tempRoomNum, tempRoomName, tempId,
                                                      tempNumOfDays, " ");
                                else
                                      roomSpecialDouble = new
SpecialService_DoubleRoom(tempRoomNum, tempRoomName, tempId,
                                                      tempNumOfDays,
findGuestID[1]);
                                Accomodation.add(roomSpecialDouble);
                          }

                          if (tempRoomName.charAt(8) == 'T') {
                                if (findGuestID.length == 1)
                                      roomSpecialTriple = new
SpecialService_TripleRoom(tempRoomNum, tempRoomName, tempId,
                                                      tempNumOfDays, " ");
                                else
```

```
                                        roomSpecialTriple = new
SpecialService_TripleRoom(tempRoomNum, tempRoomName, tempId,
                                                        tempNumOfDays,
findGuestID[1]);
                                Accomodation.add(roomSpecialTriple);
                        }
                        x++; // count increments
                }

                sc.close();

                // no file was found - which means program runs for the first
time ever
            } catch (IOException e) { // can be safely ignored
                e.getMessage();
                except = true; // flag becomes true

            }

            // if its for the first time, creation and storage of objects is
needed
            if (except == true) {
                // respective room numbers in this scenario hard coded
                int totalNumOfRooms = 30;
                int totalSingleStandard = 4;
                int totalSingleDeluxe = 3;
                int totalSinglePremium = 3;
                int totalDoubleStandard = 3;
                int totalDoubleDeluxe = 4;
                int totalDoublePremium = 3;
                int totalTripleStandard = 3;
                int totalTripleDeluxe = 3;
                int totalTriplePremium = 4;
                int SpecialDouble = 1;
                int SpecialTriple = 1;

                // starting roomNumber is 101
                int roomNumber = 101;

                // the construction and storage of 32 rooms (special rooms
included)
                // When a user tries to book or check out, their attributes
will change from
                // their initial status of not booked.
                for (int i = 0; i < totalSingleStandard; i++) {
                        roomStandard = new Standard(roomNumber, "Single-
Standard", " ", 0);
                        Accomodation.add(roomStandard);
                        roomNumber++;
                }
                for (int i = 0; i < totalSingleDeluxe; i++) {
                        roomDeluxe = new Deluxe(roomNumber, "Single-Deluxe", "
", 0);
                        Accomodation.add(roomDeluxe);
                        roomNumber++;
                }
                for (int i = 0; i < totalSinglePremium; i++) {
                        roomPremium = new Premium(roomNumber, "Single-Premium",
" ", 0);
```

```
                        Accomodation.add(roomPremium);
                        roomNumber++;
                    }
                    for (int i = 0; i < totalDoubleStandard; i++) {
                        roomStandard = new Standard(roomNumber, "Double-
Standard", " ", 0);
                        Accomodation.add(roomStandard);
                        roomNumber++;
                    }
                    for (int i = 0; i < totalDoubleDeluxe; i++) {
                        roomDeluxe = new Deluxe(roomNumber, "Double-Deluxe", "
", 0);
                        Accomodation.add(roomDeluxe);
                        roomNumber++;
                    }
                    for (int i = 0; i < totalDoublePremium; i++) {
                        roomPremium = new Premium(roomNumber, "Double-Premium",
" ", 0);
                        Accomodation.add(roomPremium);
                        roomNumber++;
                    }
                    for (int i = 0; i < totalTripleStandard; i++) {
                        roomStandard = new Standard(roomNumber, "Triple-
Standard", " ", 0);
                        Accomodation.add(roomStandard);
                        roomNumber++;
                    }
                    for (int i = 0; i < totalTripleDeluxe; i++) {
                        roomDeluxe = new Deluxe(roomNumber, "Triple-Deluxe", "
", 0);
                        Accomodation.add(roomDeluxe);
                        roomNumber++;
                    }
                    for (int i = 0; i < totalTriplePremium; i++) {
                        roomPremium = new Premium(roomNumber, "Triple-Premium",
" ", 0);
                        Accomodation.add(roomPremium);
                        roomNumber++;
                    }

                    roomSpecialDouble = new SpecialService_DoubleRoom(131,
"Special-Double", " ", 0, "  ");
                    Accomodation.add(roomSpecialDouble);
                    roomSpecialTriple = new SpecialService_TripleRoom(132,
"Special-Triple", " ", 0, "  ");
                    Accomodation.add(roomSpecialTriple);

                }
                // User interface for the resort management system
                System.out.println("* WELCOME TO RESORT BY SEA HOTEL *");
                System.out.println(" ");
                int choice = 0;// Variable to store the user's menu choice

                try {
                    // Main menu loop runs until the user selects the option to
exit
                    while (choice != 6) {
                        printMainMenu();// Display the main menu options
```

```
                        choice = main.nextInt();// Get user input for menu
choice
                        System.out.println(" ");

                        // Handle user's choice
                        switch (choice) {
                        case 1:
                                printRooms();// Display details of all rooms
                                break;

                        case 2:
                                System.out.print("Enter the Room Number to view
details : ");
                                int roomNum = main.nextInt();// Get the specific
room number
                                printSpecificRoom(roomNum, Accomodation);//
Display details of the specified room
                                break;
                        case 3:
                                bookRoom(Accomodation);// Book a room
                                break;

                        case 4:
                                checkOutRoom(Accomodation);// Check out from a
room
                                break;

                        case 5:
                                printSummary(Accomodation);// Show a summary of
room bookings
                                break;
                        case 6:
                                System.out.println("Program Terminating.....");
                                // when case 6-exit is chosen, existing data will
be written to the text file
                                PrintWriter pw = new PrintWriter(new
FileWriter("Rooms.txt"));

                                pw.println(totalSum);// Write the total sum of
bookings
                                pw.println(Accomodation.size());// Write the
number of rooms
                                for (int i = 0; i < Accomodation.size(); i++) {

        pw.println(Accomodation.get(i).getRoomNumber());

        pw.println(Accomodation.get(i).getNumberOfDaysBooked());
                                        if (Accomodation.get(i) instanceof
SpecialService_DoubleRoom) {
                                                SpecialService_DoubleRoom special =
(SpecialService_DoubleRoom) Accomodation.get(i);
                                                pw.println(special.getRoomName() +
"/" + special.getGuestID() + "#"
                                                                +
special.getSupportGuestID());
                                        } else if (Accomodation.get(i) instanceof
SpecialService_TripleRoom) {
                                                SpecialService_TripleRoom special =
(SpecialService_TripleRoom) Accomodation.get(i);
```

```
                                                pw.println(special.getRoomName() +
"/" + special.getGuestID() + "#"
                                                                        +
special.getSupportGuestID());
                                        } else

      pw.println(Accomodation.get(i).getRoomName() + "/" +
Accomodation.get(i).getGuestID());
                                }

                                pw.close();
                                break;
                        default:
                                // if user enters an invalid number
                                System.out.println(ANSI_RED + "Invalid Option,
Please Try Again" + ANSI_RESET);
                                System.out.println(" ");

                        }

                }
            } catch (InputMismatchException i) {
                // Handle input mismatch exceptions
                System.out.println(i.getMessage());
            }
        }

        // Method to display the main menu
        public class PaymentDemo {
            public static void main(String[] args) throws ParseException {
                Scanner sc = new Scanner(System.in);
                ArrayList<Payment> collection = new ArrayList<>();
                int payCount = 0;
                int option = 0;
                boolean flag = false;
                do {
                        System.out.println("1 - New Payment");
                        System.out.println("2 - Display All Payment");
                        System.out.println("3 - Search and Print Payment");
                        System.out.println("4 - Delete a Payment ");
                        System.out.println("5 - Maximum Payemnt");
                        System.out.println("6. Exit");
                        option = sc.nextInt();

        // Method to display the room menu
        public static void printRoomMenu() {
            System.out.println("-ROOM MENU-");
            System.out.println("-----------");
            System.out.println("- Single");
            System.out.println("- Double");
            System.out.println("- Triple");
            System.out.println("- Special-Service");
        }

        // Method to display the sub-room menu
        public static void printSubRoomMenu() {
            System.out.println("-SUB ROOM MENU-");
            System.out.println("---------------");
            System.out.println("- Standard");
```

```
            System.out.println("- Deluxe");
            System.out.println("- Premium");
      }

      // Method for booking a room
      // Assumes that booking and check-in occur on the same day
      public static void bookRoom(ArrayList<Room> Accomodation) {
            Scanner bookRoom = new Scanner(System.in);
            // local varaibles declaration
            boolean illegal = false;// Flag to indicate invalid input
            boolean possible = true;// Flag to check if booking is possible
            Room room = null;// Placeholder for room object
            Room printRoom = null;
            String roomSubType = null;// Variable to store room subtype
            int index = 0;
            String errorText = "";// Variable to store error messages
            public abstract class Payment {

                  protected static int paymentID=0;
                  protected int id;
                  protected double amount;

                  public Payment(double amount) {
                        this.amount = amount;
                        paymentID++;
                        id=paymentID;
                  }
            // Prompt user for their ID
            System.out.print("Enter your ID : ");
            String guestId = bookRoom.nextLine();
            System.out.println(" ");

            printRoomMenu();// Display room menu options

            // Validate user input for room type
            System.out.println(" ");
            System.out.print("Enter your desired room type : ");
            String roomType = bookRoom.nextLine();
            System.out.println(" ");

            // checks if the user keeps input space empty or enters anything out
of range
            if (!(roomType.isEmpty() || roomType.length() < 6)) {
                  // checks if it's a special room request
                  if (roomType.substring(0, 6).equalsIgnoreCase("specia")) {
                        System.out.println("Two types of Special Service Rooms
are available. Choose as per your need");
                        System.out.println("~ Double (of type Standard)");
                        System.out.println("~ Triple (of type Standard)");
                        System.out.print("Enter : ");
                        roomSubType = bookRoom.nextLine();
                        System.out.println(" ");
                  } else {
                        printSubRoomMenu();
                        System.out.print("Enter your choice : ");
                        roomSubType = bookRoom.nextLine();
                        System.out.println(" ");
                  }
            }
```

```
            String roomName = roomType + "-" + roomSubType; // Construct the
room name based on user input

            // Attempt to create a room object to validate user input
            try {
                  room = new TestRoom(0, roomName, " ", 0); // Use TestRoom to
validate room name

            } catch (IllegalArgumentException e) { // Handle invalid input
                  illegal = true; // this flag variable is used to determine if
the further steps should followed
                                                // or not depending on user's input
                  System.out.println(ANSI_RED + e.getMessage() + ANSI_RESET);
                  System.out.println(" ");

            }

            // if user's entry is legal
            if (illegal == false) {

                  // Prompt the user for the number of days they wish to book
                  System.out.print("Enter the number of days you would like to
book : ");
                  int checkInDays = bookRoom.nextInt();
                  System.out.println(" ");


                  public int getPaymentID() {
                        return paymentID;
                  }

                  public String toString() {
                        return "Payment ID: " + id + " Amount: " + amount;
                  }

                  public abstract void PaymentMethod() ;

                  public int getId() {
                        return id;
                  }

                  public double getAmount() {
                        return amount;
                  }

                  public void setAmount(double amount) {
                        this.amount = amount;
                  }


            }

                  // Iterate through the list of rooms to find a suitable
booking option
                  for (int i = 0; i < Accomodation.size(); i++) {
                        // booked = true; //initially this is made true for
each iteration
                        String[] splitRoomName =
Accomodation.get(i).getRoomName().split("-"); // stored room name splitted for
```

```
                        // Check if the room type matches the user's request//
checking prupose
                        if (splitRoomName[0].substring(0,
6).equalsIgnoreCase(roomType.substring(0, 6)) == true) {
                                // Check if the room subtype also matches
                                if (splitRoomName[1].substring(0,
6).equalsIgnoreCase(roomSubType.substring(0, 6)) == true) {
                                        // second part matches (standard, deluxe,
premium)
                                        printRoom = Accomodation.get(i); // could
be booked so stores it in the printRoom variable for

                    // later printing details
                                        index = i;

                                        // Attempt to check in to the room
                                        try {
                                                Accomodation.get(i).checkIn(guestId,
checkInDays);
                                                possible = true; // this variable
checks if although one room of a type is booked, others
                                                                                //
maybe vacant
                                                System.out.println(ANSI_GREEN +
"Room booking successful" + ANSI_RESET);
                                                System.out.println(" ");
                                                // if check-in successful, receipt
printed
                                                printReceipt(printRoom);
                                                break;// Exit loop once booking is
successful
                                        } catch (IllegalArgumentException a) {
                                                // this is thrown from the special
service rooms. They require an extra ID, if
                                                // it's not given, an exception is
thrown
                                                System.out.println(ANSI_RED +
a.getMessage() + ANSI_RESET);
                                                System.out.println("");

                                        } catch (AccomodationException e) {
                                                // this is thrown from the room
class, check in method, we don't print it right
                                                // away as the next room might be of
the same type and maybe available for
                                                // booking
                                                errorText = e.getMessage();
                                                possible = false; // at last if this
remains false, that means that type cannot be booked
                                                                                //
because they are already reserved

                                        }

                                }
                        }
                }
```

```
            }

            // here we print the error text thrown from the check in method
indicating that
            // room booking is not possible
            if (possible == false) {
                  System.out.println(ANSI_RED + errorText + ANSI_RESET);
                  System.out.println(" ");

                  // if no booking is made, this recommends the user with rooms
with a larger
                  // capacity as instructed in the assignment
                  System.out.println(BRIGHT_BLUE + "You may try one these rooms
as alternatives" + ANSI_RESET);
                  System.out.println();
                  String[] previous = Accomodation.get(index -
1).getRoomName().split("-");

                  for (int i = index; i < Accomodation.size(); i++) {
                        if (Accomodation.get(i).getNumberOfDaysBooked() == 0) {
                              String[] current =
Accomodation.get(i).getRoomName().split("-");
                              if (current[0].equalsIgnoreCase(previous[0]) ==
false) {
                                    previous =
Accomodation.get(i).getRoomName().split("-");
                                    System.out.println(

      Accomodation.get(i).getRoomName() + " | " +
Accomodation.get(i).getDescription());
                                    System.out.println(" ");

                                    continue;
                              }
                              if (current[1].equalsIgnoreCase(previous[1]) ==
false) {
                                    previous =
Accomodation.get(i).getRoomName().split("-");
                                    System.out.println(

      Accomodation.get(i).getRoomName() + " | " +
Accomodation.get(i).getDescription());
                                    System.out.println(" ");

                              }
                        }
                  }
            }

            System.out.println(" ");

      }

      public static void checkOutRoom(ArrayList<Room> Accomodation) {
            Scanner coutRoom = new Scanner(System.in);
            boolean roomFound = false;
            int index = 0;
            // prompting
            System.out.print("Enter the room number you want to check-out : ");
```

```
            int roomNum = coutRoom.nextInt();

            // initial stage where no rooms are booked but user tries to check
out a room

            // iterates through the arraylist to find the room number user
entered
            for (int i = 0; i < Accomodation.size(); i++) {
                  if (Accomodation.get(i).getRoomNumber() == roomNum) {
                        roomFound = true;
                        // if room number equals, tries to check-out by calling
check out in room class
                        try {
                              int num =
Accomodation.get(index).getNumberOfDaysBooked();
                              Accomodation.get(i).checkOut();
                              totalSum -= (Accomodation.get(index).costPerDay *
num);
                              index = i;
                              System.out.println(ANSI_GREEN + "Room " + roomNum
+ " checked out successfully" + ANSI_RESET);
                              System.out.println(" ");

                        } catch (AccomodationException e) { // exception caught
indicating that the room number user tried to

      // check out is not booked
                              e = new AccomodationException(ANSI_RED
                                          + "Sorry, the room number you are
trying to check-out is not booked :(" + ANSI_RESET);
                              System.out.println(e.getMessage());
                              System.out.println("");

                        }
                        break;
                  }
            }

            // invalid room number
            if (roomFound == false) {
                  System.out.println(ANSI_RED + "Room Number does not exit !" +
ANSI_RESET);
                  System.out.println(" ");
            }

            // coutRoom.close();
      }

      // printing booking receipt
      public static void printReceipt(Room room) {
            System.out.println("---BOOKING RECEIPT---");
            System.out.println("Guest ID : " + room.getGuestID());
            if (room instanceof SpecialService_DoubleRoom) {
                  SpecialService_DoubleRoom special =
(SpecialService_DoubleRoom) room;
                  System.out.println("Support Guest ID : " +
special.getSupportGuestID());
                  System.out.println("Ramp Length : " +
special.getRamp_length());
```

```
                    System.out.println("Ramp Width : " + special.getRamp_width());
                    System.out.println("Emergency Calling Number : " +
special.getEmergencyNum());

             } else if (room instanceof SpecialService_TripleRoom) {
                    SpecialService_TripleRoom special =
(SpecialService_TripleRoom) room;
                    System.out.println("Support Guest ID : " +
special.getSupportGuestID());
                    System.out.println("Ramp Length : " +
special.getRamp_length());
                    System.out.println("Ramp Width : " + special.getRamp_width());
                    System.out.println("Emergency Calling Number : " +
special.getEmergencyNum());
             }

             System.out.println("Room ID : " + Room.getRoomID());
             System.out.println("Room Number booked : " + room.getRoomNumber());
             System.out.println("About Room : " + room.getDescription());
             System.out.println("Number of days booked : " +
room.getNumberOfDaysBooked());
             System.out.println("Total Charge : $" +
(room.getNumberOfDaysBooked() * room.determinePrice()));
             totalSum += room.getNumberOfDaysBooked() * room.determinePrice();
             System.out.println(" ");
       }

       // printing the rooms info
       public static void printRooms() {

             System.out.println(BRIGHT_BLUE + "|--------------------------------
-------------------|");
             System.out.println("| Category | Single Room | Double Room | Triple
Room |");
             System.out.println("|          | (2 Guests)    (4 Guests)    (6
Guests)  |");
             System.out.println("|----------------------------------------------
-----|");
             System.out.println("| Standard |     $230    |     $350    |
$500    |");
             System.out.println("|  Deluxe  |     $280    |     $430    |
$600    |");
             System.out.println("| Premium  |     $350    |     $500    |
$690    |");
             System.out.println("|----------------------------------------------
-----|");
             System.out.println(" ");
             System.out.println("Standard - Offers Basic Room Amneties");
             System.out.println(" Deluxe  - Offers more space and a bathtub ");
             System.out.println("Premium  - Offers a spa and a kitchenette" +
ANSI_RESET);
             System.out.println(" ");
       }

       // printing the summary
       public static void printSummary(ArrayList<Room> Accomodation) {
             int totalCheckedIn = 0;
             boolean available = false;
```

```
            System.out.println(BRIGHT_BLUE + "--ROOM AVAILABILITY--");
            System.out.println(" ");
            System.out.println("           ***Rooms Currently Available***");
            System.out.println("-----------------------------------------------
-----" + ANSI_RESET);

            String[] previous = Accomodation.get(0).getRoomName().split("-");
            if (Accomodation.get(0).getNumberOfDaysBooked() == 0) {
                  System.out.println(Accomodation.get(0).getRoomName() + " | " +
Accomodation.get(0).getDescription());
                  System.out.println(" ");
            }
            for (int i = 0; i < Accomodation.size(); i++) {
                  if (Accomodation.get(i).getNumberOfDaysBooked() == 0) {
                        String[] current =
Accomodation.get(i).getRoomName().split("-");
                        if (current[0].equalsIgnoreCase(previous[0]) == false)
{
                              previous =
Accomodation.get(i).getRoomName().split("-");
                              System.out

      .println(Accomodation.get(i).getRoomName() + " | " +
Accomodation.get(i).getDescription());
                              System.out.println(" ");
                              available = true;
                              continue;
                        }
                        if (current[1].equalsIgnoreCase(previous[1]) == false)
{
                              previous =
Accomodation.get(i).getRoomName().split("-");
                              System.out

      .println(Accomodation.get(i).getRoomName() + " | " +
Accomodation.get(i).getDescription());
                              System.out.println(" ");
                              available = true;
                        }

                  }

            }

            if (available == false) {
                  System.out.println(ANSI_RESET + "No Room Available " +
ANSI_RESET);
            }

            available = true;


            System.out.println(" ");
            System.out.println(BRIGHT_BLUE + "  ***Rooms Currently
Occupied***");
            System.out.println("-----------------------------------" +
ANSI_RESET);
            System.out.println(" ");
```

```
            for (int i = 0; i < Accomodation.size(); i++) {
                  if (Accomodation.get(i).getNumberOfDaysBooked() != 0) {
                        System.out.println("#" +
Accomodation.get(i).getRoomNumber() + " " + Accomodation.get(i).getRoomName()
                                    + " occupied by " +
Accomodation.get(i).getGuestID());
                        totalCheckedIn++;
                        available = false;
                  }
            }
            if (available == true) {
                  System.out.println(ANSI_RESET + "No Room is booked yet " +
ANSI_RESET);
            }

            System.out.println(" ");
            System.out.println(PURPLE + "-TOTALS-");
            System.out.println("-------------------");
            System.out.println("Total Number of checked-in rooms : " +
totalCheckedIn);
            System.out.println("Total Income from the current checked-in rooms
:" + totalSum + ANSI_RESET);
            System.out.println(" ");
            System.out.println(" ");
      }
      option = sc.nextInt();
      switch (option) {
      case 1:
            if (payCount < 4) {
                  System.out.println("Select the payemnt type you wish to
consider");
                  System.out.println("1. Card");
                  System.out.println("2. Cheque");
                  int choice = sc.nextInt();
                  sc.nextLine();
                  switch (choice) {
                  case 1:
                        System.out.print("Enter Card Number : ");
                        String cardNum = sc.nextLine();
                        System.out.print("Enter Card Type (Credit/Debit) : ");
                        String cardType = sc.nextLine();
                        System.out.print("Amount : ");
                        double amount1 = sc.nextDouble();
                        CardPayment card = new CardPayment(amount1, cardNum,
cardType);
                        collection.add(card);
                        payCount++;
                        break;
                  case 2:
                        System.out.print("Enter Cheque Number : ");
                        String chequeNum = sc.nextLine();
                        System.out.print("Enter Date : ");
                        String dateString = sc.nextLine();
                        SimpleDateFormat formatter = new SimpleDateFormat("dd-
mm-yyyy");
                        Date date = formatter.parse(dateString);
                        System.out.print("Enter Bank Name :");
                        String bankName = sc.nextLine();
                        System.out.print("Amount : ");
```

```
                        double amount2 = sc.nextDouble();
                        ChequePayment cheque = new ChequePayment(amount2,
chequeNum, bankName, date);
                        collection.add(cheque);
                        payCount++;
                        break;
                    }
            } else {
                    System.out.println("Pay Collection is Full");
            }
            break;
        case 2:
            if (payCount != 0) {
                    for (int i = 0; i < payCount; i++) {
                        if (collection.get(i) instanceof CardPayment) {
                            CardPayment card = (CardPayment)
collection.get(i);
                            System.out.println(card.toString());
                        }
                        if (collection.get(i) instanceof ChequePayment) {
                            ChequePayment cheque = (ChequePayment)
collection.get(i);
                            System.out.println(cheque.toString());
                        }

                    }
            } else
                    System.out.println("Pay Collection is Empty");
            break;
        case 3:
            boolean found1 = false;
            if (payCount != 0) {
                    System.out.print("Enter PaymentID to search :");
                    int id = sc.nextInt();
                    for (int i = 0; i < payCount; i++) {
                        if (id == collection.get(i).getId()) {
                            if (collection.get(i) instanceof CardPayment) {
                                CardPayment card = (CardPayment)
collection.get(i);
                                System.out.println(card.toString());
                                found1 = true;
                            }

                            if (collection.get(i) instanceof ChequePayment) {
                                ChequePayment cheque = (ChequePayment)
collection.get(i);
                                System.out.println(cheque.toString());
                                found1 = true;
                            }
                        }

                    }
            } else
                    System.out.println("Pay Collection is Empty");

            if (found1 == false) {
                    System.out.println("Payment ID not found");
            }
            break;
```

```
	// when user request the details of a specific room
	public static void printSpecificRoom(int roomNum, ArrayList<Room>
Accomodation) {
		boolean found = false;
		System.out.println(" ");

		for (int i = 0; i < Accomodation.size(); i++) {
			if (Accomodation.get(i).getRoomNumber() == roomNum) {
				found = true;
				System.out.println(BRIGHT_BLUE + "  Room Number " +
roomNum + " Details");
				System.out.println("----------------------------------
" + ANSI_RESET);
				System.out.println("Type : " +
Accomodation.get(i).getRoomName());
				System.out.println("About Room : " +
Accomodation.get(i).getDescription());
				if (Accomodation.get(i).getGuestID() != " ") {
					System.out.println("Currently occupied by Guest
ID " + Accomodation.get(i).getGuestID() + " for "
						+
Accomodation.get(i).getNumberOfDaysBooked() + " days.");

				} else {
					System.out.println("Available for booking");

				}
				System.out.println(" ");
				break;
			}
		}

		if (found == false) {
			System.out.println(ANSI_RED + "Such Room Number does not
exist" + ANSI_RESET);
			System.out.println(" ");
		}
	}

	// this is invoked from the special room class to obtain the second guest
ID
	public static String specialResponse() {
		Scanner sr = new Scanner(System.in);
		System.out.println("We require one more ID to book a special-service
room ");

		System.out.print("Enter support ID : ");
		String supportGuestId = sr.nextLine();
		System.out.println(" ");
		return supportGuestId;
	}

}
```

Room.java

```
package com.assignment2.stageD;

import com.finalTest.codes.CardPayment;
import com.finalTest.codes.ChequePayment;

// Abstract class representing a room in the accommodation system.
// This class defines common attributes and methods for different types of rooms.
public abstract class Room {

	// Unique identifier for the room
	protected int roomNumber;

	// Name of the room, which indicates its type and category (e.g., Single-
St,
	// Double-De)
	protected String roomName;

	// Description providing details about the room
	protected String description;

	// Cost of the room per day
	protected double costPerDay;

	// Static ID for the room type, shared across all instances of Room
	protected static String roomID;

	// ID of the guest currently occupying the room
	protected String guestID;

	// Number of days the room has been booked
	protected int numberOfDaysBooked;

	// Constructor to initialize a Room object with the provided parameters.
	// Throws an IllegalArgumentException if the room type is invalid.
	public Room(int roomNumber, String roomName, String guestID, int
numberOfDaysBooked)
			throws IllegalArgumentException {
		boolean valid = false;

		// Validate the roomName to ensure it meets the expected format
		if (roomName.isEmpty() || roomName.length() < 6) {
			throw new IllegalArgumentException("Room name cannot be empty
and must be at least 6 characters long.");
		}

		// Validate the roomName format based on the type (Single, Double,
Triple) and
		// category (St, De, Pr)
		switch (roomName.substring(0, 6).toLowerCase()) {
		case "single":
			// Validate the category for Single rooms
			switch (roomName.substring(7, 9).toLowerCase()) {
			case "st":
			case "de":
			case "pr":
				valid = true;
				break;
```

```
                }
                break;
            case "double":
                // Validate the category for Double rooms
                switch (roomName.substring(7, 9).toLowerCase()) {
                case "st":
                case "de":
                case "pr":
                    valid = true;
                    break;
                }
                break;
            case "triple":
                // Validate the category for Triple rooms
                switch (roomName.substring(7, 9).toLowerCase()) {
                case "st":
                case "de":
                case "pr":
                    valid = true;
                    break;
                }
                break;
            case 4:
                boolean found2 = false;
                if (payCount != 0) {
                    System.out.print("Enter Payment ID to delete :");
                    int id = sc.nextInt();
                    for (int i = 0; i <= payCount; i++) {
                        if (id == collection.get(i).getId()) {
                            collection.remove(i);
                            payCount--;
                            found2 = true;
                        }

                    }
                } else
                    System.out.println("Pay Collection is Empty");

                if (found2 == false) {
                    System.out.println("The Payment ID you entered does not
exist");
                }
                break;
            case 5:
                if (payCount != 0) {
                    int index=0;
                    double maxAmount = collection.get(0).getAmount();
                    for (int i = 1; i < payCount; i++) {
                        if(collection.get(i).getAmount()>maxAmount) {
                            maxAmount=collection.get(i).getAmount();
                            index=i;
                        }
                    }
                    if (collection.get(index) instanceof CardPayment) {
                        CardPayment card = (CardPayment)
collection.get(index);
                        System.out.println(card.toString());

                    }
```

```
                        if (collection.get(index) instanceof ChequePayment) {
                                ChequePayment cheque = (ChequePayment)
collection.get(index);
                                System.out.println(cheque.toString());

                        }

                } else
                        System.out.println("Pay Collection is Empty");
                break;
            case 6:
                System.out.println("Bye...");
            }
        } while (option != 6);
}

}

            // If the room name is valid, initialize the room attributes
            if (valid) {
                this.roomNumber = roomNumber;
                this.roomName = roomName;
                this.guestID = guestID;
                this.numberOfDaysBooked = numberOfDaysBooked;
            } else {
                throw new IllegalArgumentException("Invalid room type
specified.");
            }
        }

        // Getter for the static roomID
        public static String getRoomID() {
            return roomID;
        }

        // Getter for the room number
        public int getRoomNumber() {
            return roomNumber;
        }

        // Setter for the guest ID
        public void setGuestID(String guestID) {
            this.guestID = guestID;
        }

        // Getter for the guest ID
        public String getGuestID() {
            return guestID;
        }

        // Getter for the number of days the room has been booked
        public int getNumberOfDaysBooked() {
            return numberOfDaysBooked;
        }

        // Setter for the number of days the room has been booked
        public void setNumberOfDaysBooked(int numberOfDaysBooked) {
            this.numberOfDaysBooked = numberOfDaysBooked;
```

```
	}

	// Getter for the room name
	public String getRoomName() {
		return roomName;
	}

	// Getter for the room description
	public String getDescription() {
		return description;
	}

	// Abstract method to determine the price per day for the room
	// This method must be implemented by subclasses of Room
	public abstract double determinePrice();

	// Method to check in a guest into the room
	// Throws an AccomodationException if the room is already booked
	public void checkIn(String guestId, int numberOfDaysBooked) throws
AccomodationException {
		if (this.getNumberOfDaysBooked() != 0) {
			throw new AccomodationException("The room is already
booked.");
		} else {
			this.setNumberOfDaysBooked(numberOfDaysBooked);
			this.setGuestID("RBS-" + guestId); // Prefix guest ID with
"RBS-"
		}
	}

	// Method to check out the guest from the room
	// Throws an AccomodationException if the room is not currently booked
	public void checkOut() throws AccomodationException {
		if (this.getNumberOfDaysBooked() == 0) {
			throw new AccomodationException("The room is not booked.");
		} else {
			this.setNumberOfDaysBooked(0); // Reset number of days booked
			this.setGuestID(""); // Clear guest ID
		}
	}
}
```

Standard.java

```
package com.assignment2.stageD;

// This class represents a Standard Room, which is a basic room type without
additional amenities.
// It extends the Room class and provides specific details and pricing for
standard rooms.
public final class Standard extends Room {
```

```
	// Constructor for initializing a Standard room object
	// Throws an IllegalArgumentException if the provided room details are
invalid
	public Standard(int roomNumber, String roomName, String guestID, int
numberOfDaysBooked)
		throws IllegalArgumentException {
		// Call the constructor of the superclass (Room) to set common room
attributes
		super(roomNumber, roomName, guestID, numberOfDaysBooked);

		// Determine the description and room ID based on the room name
		switch (roomName.charAt(0)) {
		case 'S':
			// Set description and ID for Single Standard Room
			this.description = "Single-Standard room accommodates up to 2
guests with basic amenities.";
			Room.roomID = "SS";
			break;
		case 'D':
			// Set description and ID for Double Standard Room
			this.description = "Double-Standard room accommodates up to 4
guests with basic amenities.";
			Room.roomID = "SD";
			break;
		case 'T':
			// Set description and ID for Triple Standard Room
			this.description = "Triple-Standard room accommodates up to 6
guests with basic amenities.";
			Room.roomID = "ST";
			break;
		default:
			// If the room name does not match any expected pattern, throw
an exception
			throw new IllegalArgumentException("Invalid room name
format.");
		}
	}

	@Override
	// Method to calculate the price per day for the Standard room
	// Returns the daily cost based on the type of Standard room
	public double determinePrice() {
		// Determine and set the cost per day based on the room type
		switch (this.getRoomName().substring(0, 6).toLowerCase()) {
		case "single":
			// Set cost for Single Standard Room
			this.costPerDay = 230;
			return 230;

		case "double":
			// Set cost for Double Standard Room
			this.costPerDay = 350;
			return 350;

		case "triple":
			// Set cost for Triple Standard Room
			this.costPerDay = 500;
			return 500;
```

```
            default:
                // Return 0 if the room type does not match any known type
                return 0;
            }
        }
}
```

Deluxe.java

```
package com.assignment2.stageD;

// The Deluxe class represents a type of room that extends the Room class.
// It is marked as final, so it cannot be subclassed.
public final class Deluxe extends Room {

    // Constructor for the Deluxe class. It initializes the room details
    // and sets the room description based on the room type.
    public Deluxe(int roomNumber, String roomName, String guestID, int
numberOfDaysBooked)
            throws IllegalArgumentException {
        super(roomNumber, roomName, guestID, numberOfDaysBooked); // Calls the
constructor of the superclass (Room)

        // Set the room description and ID based on the first character of the
room name
        switch(roomName.charAt(0)) {
            case 'S': // If the room is a Single-Deluxe
                this.description = "Single-Deluxe room can accommodate up to 2
guests, offers more space, and includes a bathtub.";
                Room.roomID = "DS"; // Set the room ID for Single-Deluxe
                break;
            case 'D': // If the room is a Double-Deluxe
                this.description = "Double-Deluxe room can accommodate up to 4
guests, offers more space, and includes a bathtub.";
                Room.roomID = "DD"; // Set the room ID for Double-Deluxe
                break;
            case 'T': // If the room is a Triple-Deluxe
                this.description = "Triple-Deluxe room can accommodate up to 6
guests, offers more space, and includes a bathtub.";
                Room.roomID = "DT"; // Set the room ID for Triple-Deluxe
                break;
        }
    }

    // Method to determine the price per day based on the room type
    @Override
    public double determinePrice() {
        // Determine the price based on the room type by checking the first part
of the room name
        switch(this.getRoomName().substring(0, 6).toLowerCase()) {
            case "single": // For Single-Deluxe room
                this.costPerDay = 280;
                return 280;
```

```
            case "double": // For Double-Deluxe room
                this.costPerDay = 430;
                return 430;

            case "triple": // For Triple-Deluxe room
                this.costPerDay = 600;
                return 600;

            default: // If the room type doesn't match, return 0
                return 0;
        }
    }
}
```

Premium.java

```
package com.assignment2.stageD;

// The Premium class extends the Room class and represents a premium category room
type.
// It is declared as final to prevent further subclassing.
public final class Premium extends Room {

      // Constructor for the Premium class. Initializes the room with specific
details
      // and
      // sets the room description and ID based on the type of premium room.
      public Premium(int roomNumber, String roomName, String guestID, int
numberOfDaysBooked)
                  throws IllegalArgumentException {
            super(roomNumber, roomName, guestID, numberOfDaysBooked); //
Initialize the base class (Room) with provided

                              // parameters

            // Set the room description and ID based on the initial character of
the room
            // name
            switch (roomName.charAt(0)) {
            case 'S': // Single-Premium room
                  this.description = "Single-Premium room accommodates up to 2
guests and includes a spa and a kitchenette.";
                  Room.roomID = "PS"; // Unique room ID for Single-Premium
                  break;
            case 'D': // Double-Premium room
                  this.description = "Double-Premium room accommodates up to 4
guests and includes a spa and a kitchenette.";
                  Room.roomID = "PD"; // Unique room ID for Double-Premium
                  break;
            case 'T': // Triple-Premium room
                  this.description = "Triple-Premium room accommodates up to 6
guests and includes a spa and a kitchenette.";
                  Room.roomID = "PT"; // Unique room ID for Triple-Premium
```

```
                    break;
             default:
                    throw new IllegalArgumentException("Invalid room name
provided."); // Handle unexpected room names
             }
       }

       // Method to calculate the price per day for the premium room based on its
type
       @Override
       public double determinePrice() {
             // Determine the price by checking the first part of the room name
             switch (this.getRoomName().substring(0, 6).toLowerCase()) {
             case "single": // Price for Single-Premium room
                    this.costPerDay = 350;
                    return 350;

             case "double": // Price for Double-Premium room
                    this.costPerDay = 500;
                    return 500;

             case "triple": // Price for Triple-Premium room
                    this.costPerDay = 690;
                    return 690;

             default: // Handle cases where the room type is unknown
                    return 0;
             }
       }
}
```

TestRoom.java

```
package com.assignment2.stageD;

// This class is a test implementation of the Room class.
// It provides a basic implementation for testing purposes and overrides the
determinePrice method.
public class TestRoom extends Room {
```

```
	// Constructor for initializing a TestRoom object
	// Calls the constructor of the superclass (Room) to set the common
attributes
	// of the room
	public TestRoom(int roomNumber, String roomName, String guestID, int
numberOfDaysBooked)
			throws IllegalArgumentException {
		// Initialize the Room with the provided parameters
		super(roomNumber, roomName, guestID, numberOfDaysBooked);
	}

	@Override
	// Method to calculate the price per day for the TestRoom
	// In this test implementation, the method returns 0 as a placeholder
	public double determinePrice() {
		// Returning 0 as this is a test implementation
		return 0;
	}
}
```

SpecialServices.java

```
package com.assignment2.stageD;

public interface Special_Services {

	final String category = "Standard";
	final String emergency_num = "077-4753244";

	public String getEmergencyNum();

}
```

SpecialService_DoubleRoom.java

```
package com.assignment2.stageD;

// Class representing a Special Service Double Room, which includes additional
features
// such as ramps and requires a support guest ID. This class extends Room and
implements
// Special_Services interface.
```

```
public final class SpecialService_DoubleRoom extends Room implements
Special_Services {

	// Fixed dimensions for the ramp associated with this room
	final double ramp_length = 100; // Length of the ramp in centimeters
	final double ramp_width = 30; // Width of the ramp in centimeters

	// ID of the support guest who accompanies the primary guest
	String supportGuestID;

	// Constructor to initialize a SpecialService_DoubleRoom object
	// Throws an IllegalArgumentException if the provided room details are
invalid
	public SpecialService_DoubleRoom(int roomNumber, String roomName, String
guestID, int numberOfDaysBooked,
			String supportGuestId) throws IllegalArgumentException {
		// Call the constructor of the superclass (Room)
		super(roomNumber, roomName, guestID, numberOfDaysBooked);

		// Initialize the support guest ID
		this.supportGuestID = supportGuestId;

		// Set specific attributes for this type of room
		this.roomNumber = roomNumber;
		this.description = "Special-Service room equipped with ramps for
accessibility";
		Room.roomID = "*SSD1"; // Unique room ID for this room type
	}

	@Override
	// Method to set the number of days booked for the room
	// This method overrides the superclass method to include additional logic
for
	// special service rooms
	public void setNumberOfDaysBooked(int numberOfDaysBooked) {
		// Prompt for the support guest ID if not already provided
		this.supportGuestID = StageD.specialResponse();

		// Check if a valid support guest ID has been entered
		if (supportGuestID.isEmpty()) {
			// If no ID is provided, throw an exception
			throw new IllegalArgumentException("Failed to book special-
service room. Support guest ID is required.");
		} else {
			// If valid, set the number of days booked
			this.numberOfDaysBooked = numberOfDaysBooked;
		}
	}

	@Override
	// Method to determine the price of the room per day
	public double determinePrice() {
		// Set and return the cost per day for a Special-Service Double Room
		this.costPerDay = 350;
		return 350;
	}

	@Override
	// Method to get the emergency contact number
```

```
	public String getEmergencyNum() {
		// Return the emergency number for this room type
		// Note: Ensure that emergency_num is defined in the class or
interface
		return emergency_num;
	}

	// Getter for the ramp length
	public double getRamp_length() {
		return ramp_length;
	}

	// Getter for the ramp width
	public double getRamp_width() {
		return ramp_width;
	}

	// Getter for the support guest ID
	public String getSupportGuestID() {
		return supportGuestID;
	}
}
```

SpecialService_TripleRoom.java

```
package com.assignment2.stageD;

// This class represents a Special Service Triple Room which includes additional
features
// such as ramps for accessibility and requires a support guest ID. It extends the
Room class
// and implements the Special_Services interface.
public final class SpecialService_TripleRoom extends Room implements
Special_Services {

	// Fixed dimensions for the ramp associated with this room
	final double ramp_length = 90; // Length of the ramp in centimeters
	final double ramp_width = 35; // Width of the ramp in centimeters

	// ID of the support guest who accompanies the primary guest
	String supportGuestID;

	// Constructor to initialize a SpecialService_TripleRoom object
	// Throws an IllegalArgumentException if the provided room details are
invalid
	public SpecialService_TripleRoom(int roomNumber, String roomName, String
guestID, int numberOfDaysBooked,
			String supportGuestId) throws IllegalArgumentException {
		// Call the constructor of the superclass (Room)
		super(roomNumber, roomName, guestID, numberOfDaysBooked);

		// Initialize the support guest ID
		this.supportGuestID = supportGuestId;
```

```
            // Set specific attributes for this type of room
            this.roomNumber = roomNumber;
            this.description = "Special-Service room equipped with ramps for
accessibility";
            Room.roomID = "*SST1"; // Unique room ID for this room type
      }

      @Override
      // Method to set the number of days booked for the room
      // This method overrides the superclass method to include additional logic
for
      // special service rooms
      public void setNumberOfDaysBooked(int numberOfDaysBooked) {
            // Prompt for the support guest ID if not already provided
            this.supportGuestID = StageD.specialResponse();

            // Check if a valid support guest ID has been entered
            if (supportGuestID.isEmpty()) {
                  // If no ID is provided, throw an exception indicating failure
to book
                  throw new IllegalArgumentException(
                              "Booking failed: Support guest ID is required for
special-service rooms.");
            } else {
                  // If valid, set the number of days booked
                  this.numberOfDaysBooked = numberOfDaysBooked;
            }
      }

      @Override
      // Method to determine the daily price of the room
      public double determinePrice() {
            // Set and return the cost per day for a Special-Service Triple Room
            this.costPerDay = 500;
            return 500;
      }

      @Override
      // Method to get the emergency contact number for this room type
      public String getEmergencyNum() {
            // Return the emergency number for this room type
            // Ensure that emergency_num is defined and initialized
appropriately
            return emergency_num;
      }

      // Getter method for the ramp length
      public double getRamp_length() {
            return ramp_length;
      }

      // Getter method for the ramp width
      public double getRamp_width() {
            return ramp_width;
      }

      // Getter method for the support guest ID
      public String getSupportGuestID() {
            return supportGuestID;
```

```
	}

}
```

AccomodationException.java

```
package com.assignment2.stageD;

// Custom exception class to handle accommodation-related errors
@SuppressWarnings("serial")
public class AccomodationException extends Exception {

	// Default error message when all rooms of a specific type are booked
	private String message = "All rooms of the type you selected are already
booked!";

	// Default constructor that calls the superclass constructor
	public AccomodationException() {
		super(); // Calls the Exception class's constructor
	}

	// Constructor that allows setting a custom error message
	public AccomodationException(String message) {
		super(message); // Passes the custom message to the Exception
class's constructor
		this.message = message; // Assigns the custom message to the class's
message field
	}

	// Override of the getMessage method to return the custom error message
	@Override
	public String getMessage() {
		return message; // Returns the message that was set in the
constructor
	}
}
```

OUTPUTS

Invalid entry in main menu

```
Console ×
StageD [Java Application] C:\Program Files\Java\jdk-22\bin\javaw.exe  (18 Aug 2024, 10:59:38) [pid: 14916]
* WELCOME TO RESORT BY SEA HOTEL *

-MAIN MENU-
-----------
1. Display All Rooms' Details
2. Check Details of a specific room
3. Check-In
4. Check-Out
5. Show Summary
6. Exit
Enter your choice : 7

Invalid Option, Please Try Again

-MAIN MENU-
-----------
1. Display All Rooms' Details
2. Check Details of a specific room
3. Check-In
4. Check-Out
5. Show Summary
6. Exit
Enter your choice :
```

Displaying All Room Information

```
Console ×
StageD [Java Application] C:\Program Files\Java\jdk-22\bin\javaw.exe  (18 Aug 2024, 11:01:09) [pid: 10940]
* WELCOME TO RESORT BY SEA HOTEL *

-MAIN MENU-
-----------
1. Display All Rooms' Details
2. Check Details of a specific room
3. Check-In
4. Check-Out
5. Show Summary
6. Exit
Enter your choice : 1
|
|--------------------------------------------------|
| Category | Single Room | Double Room | Triple Room |
|          | (2 Guests)    (4 Guests)    (6 Guests)  |
|--------------------------------------------------|
| Standard |     $230    |     $350    |     $500    |
|  Deluxe  |     $280    |     $430    |     $600    |
| Premium  |     $350    |     $500    |     $690    |
|--------------------------------------------------|

Standard - Offers Basic Room Amneties
 Deluxe  - Offers more space and a bathtub
Premium  - Offers a spa and a kitchenette

-MAIN MENU-
-----------
1. Display All Rooms' Details
2. Check Details of a specific room
3. Check-In
4. Check-Out
5. Show Summary
6. Exit
Enter your choice :
```

Displaying Details of a specific room (ALL conditions below shown in one Screenshot)

1. A room that is booked
2. A room that is not booked
3. Trying to list a room that doesn't exit

```
Console ×
StageD [Java Application] C:\Program Files\Java\jdk-22\bin\javaw.exe  (18 Aug 2024, 11:03:05) [pid: 4852]

-MAIN MENU-
-----------
1. Display All Rooms' Details
2. Check Details of a specific room
3. Check-In
4. Check-Out
5. Show Summary
6. Exit
Enter your choice : 2

Enter the Room Number to view details : 101

  Room Number 101 Details
-----------------------------------
Type : Single-Standard
About Room : Single-Standard room can fit upto 2 guests and has basic room amneties
Available for booking

-MAIN MENU-
-----------
1. Display All Rooms' Details
2. Check Details of a specific room
3. Check-In
4. Check-Out
5. Show Summary
6. Exit
Enter your choice : 2

Enter the Room Number to view details : 200

Such Room Number does not exist

-MAIN MENU-
-----------
1. Display All Rooms' Details
2. Check Details of a specific room
3. Check-In
4. Check-Out
5. Show Summary
6. Exit
Enter your choice :
```

Checking In

1. Successful Checking-In

```
Console ×
StageD [Java Application] C:\Program Files\Java\jdk-22\bin\javaw.exe (18 Aug 2024, 11:04:54) [pid: 7724]
* WELCOME TO RESORT BY SEA HOTEL *

-MAIN MENU-
-----------
1. Display All Rooms' Details
2. Check Details of a specific room
3. Check-In
4. Check-Out
5. Show Summary
6. Exit
Enter your choice : 3

Enter your ID : banuka2004

-ROOM MENU-
-----------
- Single
- Double
- Triple
- Special-Service

Enter your desired room type : Single

-SUB ROOM MENU-
---------------
- Standard
- Deluxe
- Premium
Enter your choice : Premium

Enter the number of days you would like to book : 2

Room booking successful

---BOOKING RECEIPT---
Guest ID : RBS-banuka2004
Room ID : *SST1
Room Number booked : 108
About Room : Single-Premium room can fit upto 2 guests and has a spa and a kitchenette
Number of days booked : 2
Total Charge : $700.0
```

2. Invalid Room Type Error

```
Console ×
StageD [Java Application] C:\Program Files\Java\jdk-22\bin\javaw.exe  (18 Aug 2024, 11:08:04) [pid: 6576]
* WELCOME TO RESORT BY SEA HOTEL *

-MAIN MENU-
-----------
1. Display All Rooms' Details
2. Check Details of a specific room
3. Check-In
4. Check-Out
5. Show Summary
6. Exit
Enter your choice : 3

Enter your ID : KTRS24

-ROOM MENU-
-----------
- Single
- Double
- Triple
- Special-Service

Enter your desired room type : ererr

Invalid Room Type !


-MAIN MENU-
-----------
1. Display All Rooms' Details
2. Check Details of a specific room
3. Check-In
4. Check-Out
5. Show Summary
6. Exit
Enter your choice :
```

3. Trying to book a room of type that all of that type has been booked
   - Displaying Recommendations of larger capacity

Scenario 1

```
Console ×
StageD [Java Application] C:\Program Files\Java\jdk-22\bin\javaw.exe  (18 Aug 2024, 11:12:01) [pid: 1896]

-MAIN MENU-
-----------
1. Display All Rooms' Details
2. Check Details of a specific room
3. Check-In
4. Check-Out
5. Show Summary
6. Exit
Enter your choice : 3

Enter your ID : 9876V

-ROOM MENU-
-----------
- Single
- Double
- Triple
- Special-Service

Enter your desired room type : Single

-SUB ROOM MENU-
---------------
- Standard
- Deluxe
- Premium
Enter your choice : Premium

Enter the number of days you would like to book : 1

All rooms of the type you selected are already booked !

You may try one these rooms as alternatives

Double-Standard | Double-Standard room can fit upto 4 guests and has basic room amneties

Double-Deluxe | Double-Deluxe room can fit upto 4 guests but is more spacious and has a bathtub

Double-Premium | Double-Premium room can fit upto 4 guests and has a spa and a kitchenette

Triple-Standard | Triple-Standard room can fit upto 6 guests and has basic room amneties
```

```
Console
StageD [Java Application] C:\Program Files\Java\jdk-22\bin\javaw.exe  (18 Aug 2024, 11:12:01) [pid: 1896]

Enter your desired room type : Single

-SUB ROOM MENU-
---------------
- Standard
- Deluxe
- Premium
Enter your choice : Premium

Enter the number of days you would like to book : 1

All rooms of the type you selected are already booked !

You may try one these rooms as alternatives

Double-Standard | Double-Standard room can fit upto 4 guests and has basic room amneties

Double-Deluxe | Double-Deluxe room can fit upto 4 guests but is more spacious and has a bathtub

Double-Premium | Double-Premium room can fit upto 4 guests and has a spa and a kitchenette

Triple-Standard | Triple-Standard room can fit upto 6 guests and has basic room amneties

Triple-Deluxe | Triple-Deluxe room can fit upto 6 guests but is more spacious and has a bathtub

Triple-Premium | Triple-Premium room can fit upto 6 guests and has a spa and a kitchenette

Special-Double | Special-Service room fitted with ramps

Special-Triple | Special-Service room fitted with ramps
```

Scenario 2

```
Console ×
StageD [Java Application] C:\Program Files\Java\jdk-22\bin\javaw.exe  (18 Aug 2024, 11:12:01) [pid: 1896]
-MAIN MENU-
-----------
1. Display All Rooms' Details
2. Check Details of a specific room
3. Check-In
4. Check-Out
5. Show Summary
6. Exit
Enter your choice : 3

Enter your ID : HAJU123

-ROOM MENU-
-----------
- Single
- Double
- Triple
- Special-Service

Enter your desired room type : special service

Two types of Special Service Rooms are available. Choose as per your need
~ Double (of type Standard)
~ Triple (of type Standard)
Enter : double

Enter the number of days you would like to book : 1

All rooms of the type you selected are already booked !

You may try one these rooms as alternatives

Special-Triple | Special-Service room fitted with ramps


-MAIN MENU-
-----------
1. Display All Rooms' Details
2. Check Details of a specific room
3. Check-In
4. Check-Out
5. Show Summary
```

4. Booking a special service room successfully

```
Console ×
StageD [Java Application] C:\Program Files\Java\jdk-22\bin\javaw.exe  (18 Aug 2024, 11:21:53) [pid: 8408]
1. Display All Rooms' Details
2. Check Details of a specific room
3. Check-In
4. Check-Out
5. Show Summary
6. Exit
Enter your choice : 3

Enter your ID : 57767D

-ROOM MENU-
-----------
- Single
- Double
- Triple
- Special-Service

Enter your desired room type : special service

Two types of Special Service Rooms are available. Choose as per your need
~ Double (of type Standard)
~ Triple (of type Standard)
Enter : Double

Enter the number of days you would like to book : 2

We require one more ID to book a special-service room
Enter support ID : 3456B

Room booking successful

---BOOKING RECEIPT---
Guest ID : RBS-57767D
Support Guest ID : 3456B
Ramp Length : 100.0
Ramp Width : 30.0
Emergency Calling Number : 071-4489727
Room ID : *SST1
Room Number booked : 131
About Room : Special-Service room fitted with ramps
Number of days booked : 2
Total Charge : $700.0
```

5. Trying to book a special service room without entering a second guestID

```
Console ×
StageD [Java Application] C:\Program Files\Java\jdk-22\bin\javaw.exe  (18 Aug 2024, 11:21:53) [pid: 8408]
-MAIN MENU-
-----------
1. Display All Rooms' Details
2. Check Details of a specific room
3. Check-In
4. Check-Out
5. Show Summary
6. Exit
Enter your choice : 3

Enter your ID : 28643J

-ROOM MENU-
-----------
- Single
- Double
- Triple
- Special-Service

Enter your desired room type : special service

Two types of Special Service Rooms are available. Choose as per your need
~ Double (of type Standard)
~ Triple (of type Standard)
Enter : Triple

Enter the number of days you would like to book : 1

We require one more ID to book a special-service room
Enter support ID :

Failed to book special-service room


-MAIN MENU-
-----------
1. Display All Rooms' Details
2. Check Details of a specific room
3. Check-In
4. Check-Out
5. Show Summary
6. Exit
```

Checking Out (ALL conditions below shown in one Screenshot)

1. Successful Checking-Out
2. Trying to check out a room that hasn't been booked
3. Trying to access a room number that does not exist

```
Console
StageD [Java Application] C:\Program Files\Java\jdk-22\bin\javaw.exe  (18 Aug 2024, 11:21:53) [pid: 8408]
  ***Rooms Currently Occupied***
-----------------------------------

#108 Single-Premium occupied by RBS-Banuka2004
#131 Special-Double occupied by RBS-57767D

-TOTALS-
-------------------
Total Number of checked-in rooms : 2
Total Income from the current checked-in rooms :1400.0


-MAIN MENU-
-----------
1. Display All Rooms' Details
2. Check Details of a specific room
3. Check-In
4. Check-Out
5. Show Summary
6. Exit
Enter your choice : 4

Enter the room number you want to check-out : 108
Room 108 checked out successfully

-MAIN MENU-
-----------
1. Display All Rooms' Details
2. Check Details of a specific room
3. Check-In
4. Check-Out
5. Show Summary
6. Exit
Enter your choice : 4

Enter the room number you want to check-out : 107
Sorry, the room number you are trying to check-out is not booked :(

-MAIN MENU-
-----------
1. Display All Rooms' Details
```

```
Console ×
StageD [Java Application] C:\Program Files\Java\jdk-22\bin\javaw.exe  (18 Aug 2024, 11:21:53) [pid: 8408]
5. Show Summary
6. Exit
Enter your choice : 4

Enter the room number you want to check-out : 108
Room 108 checked out successfully

-MAIN MENU-
-----------
1. Display All Rooms' Details
2. Check Details of a specific room
3. Check-In
4. Check-Out
5. Show Summary
6. Exit
Enter your choice : 4

Enter the room number you want to check-out : 107
Sorry, the room number you are trying to check-out is not booked :(

-MAIN MENU-
-----------
1. Display All Rooms' Details
2. Check Details of a specific room
3. Check-In
4. Check-Out
5. Show Summary
6. Exit
Enter your choice : 4

Enter the room number you want to check-out : 200
Room Number does not exit !

-MAIN MENU-
-----------
1. Display All Rooms' Details
2. Check Details of a specific room
3. Check-In
4. Check-Out
5. Show Summary
6. Exit
Enter your choice :
```

Showing Summary

```
Console ×
StageD [Java Application] C:\Program Files\Java\jdk-22\bin\javaw.exe  (18 Aug 2024, 11:21:53) [pid: 8408]
-MAIN MENU-
-----------
1. Display All Rooms' Details
2. Check Details of a specific room
3. Check-In
4. Check-Out
5. Show Summary
6. Exit
Enter your choice : 5

--ROOM AVAILABILITY--

           ***Rooms Currently Available***
-----------------------------------------------------
Single-Standard | Single-Standard room can fit upto 2 guests and has basic room amneties

Single-Deluxe | Single-Deluxe room can fit upto 2 guests but is more spacious and has a bathtub

Single-Premium | Single-Premium room can fit upto 2 guests and has a spa and a kitchenette

Double-Standard | Double-Standard room can fit upto 4 guests and has basic room amneties

Double-Deluxe | Double-Deluxe room can fit upto 4 guests but is more spacious and has a bathtub

Double-Premium | Double-Premium room can fit upto 4 guests and has a spa and a kitchenette

Triple-Standard | Triple-Standard room can fit upto 6 guests and has basic room amneties

Triple-Deluxe | Triple-Deluxe room can fit upto 6 guests but is more spacious and has a bathtub

Triple-Premium | Triple-Premium room can fit upto 6 guests and has a spa and a kitchenette

Special-Triple | Special-Service room fitted with ramps


  ***Rooms Currently Occupied***
----------------------------------

#131 Special-Double occupied by RBS-57767D

-TOTALS-
--------------------
```

```
Console ×
StageD [Java Application] C:\Program Files\Java\jdk-22\bin\javaw.exe  (18 Aug 2024, 11:21:53) [pid: 8408]
2. Check Details of a specific room
3. Check-In
4. Check-Out
5. Show Summary
6. Exit
Enter your choice : 5

--ROOM AVAILABILITY--

          ***Rooms Currently Available***
-----------------------------------------------------
Single-Standard | Single-Standard room can fit upto 2 guests and has basic room amneties

Single-Deluxe | Single-Deluxe room can fit upto 2 guests but is more spacious and has a bathtub

Single-Premium | Single-Premium room can fit upto 2 guests and has a spa and a kitchenette

Double-Standard | Double-Standard room can fit upto 4 guests and has basic room amneties

Double-Deluxe | Double-Deluxe room can fit upto 4 guests but is more spacious and has a bathtub

Double-Premium | Double-Premium room can fit upto 4 guests and has a spa and a kitchenette

Triple-Standard | Triple-Standard room can fit upto 6 guests and has basic room amneties

Triple-Deluxe | Triple-Deluxe room can fit upto 6 guests but is more spacious and has a bathtub

Triple-Premium | Triple-Premium room can fit upto 6 guests and has a spa and a kitchenette

Special-Triple | Special-Service room fitted with ramps


  ***Rooms Currently Occupied***
----------------------------------

#131 Special-Double occupied by RBS-57767D

-TOTALS-
-------------------
Total Number of checked-in rooms : 1
Total Income from the current checked-in rooms :1400.0
```

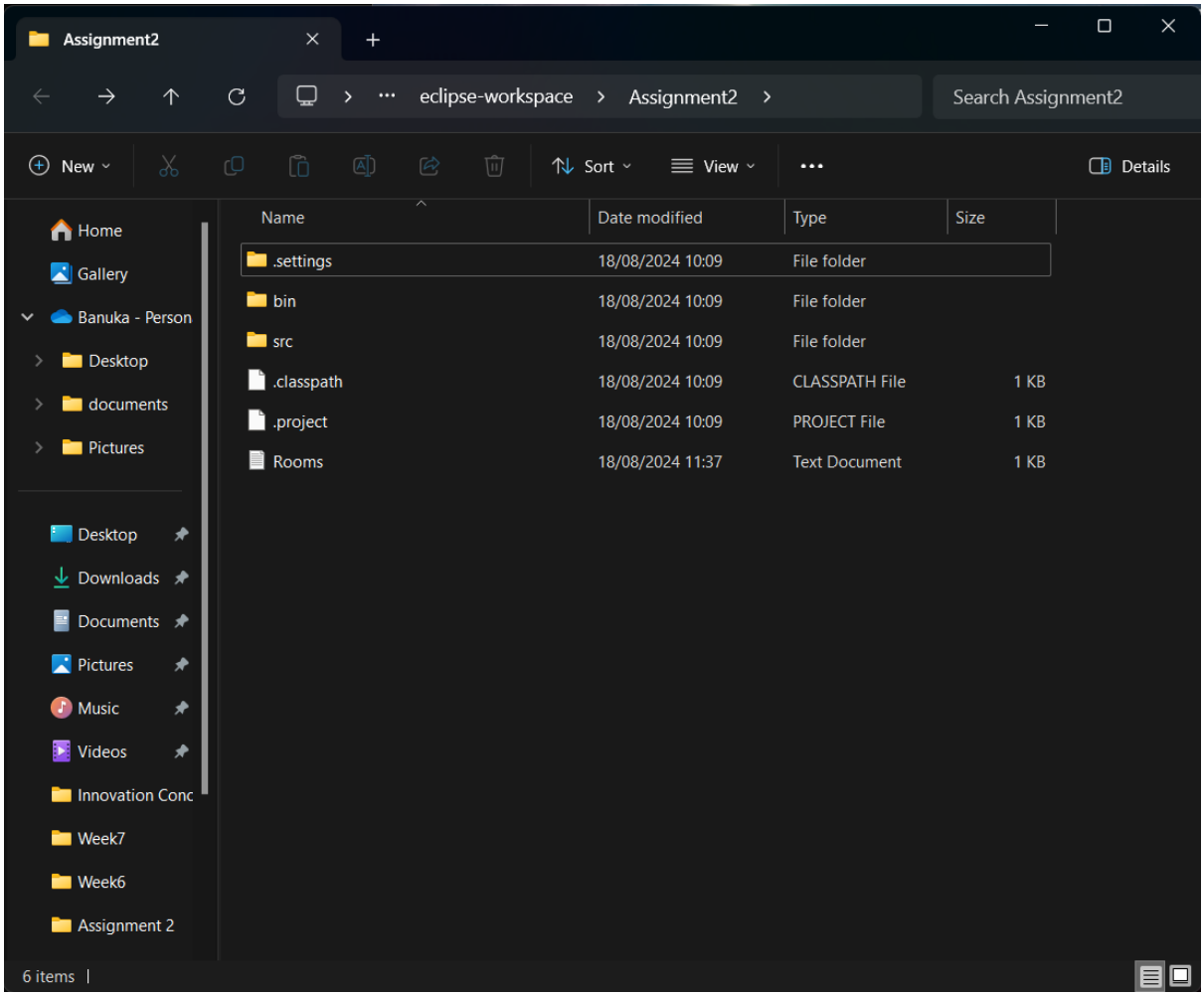
Exit

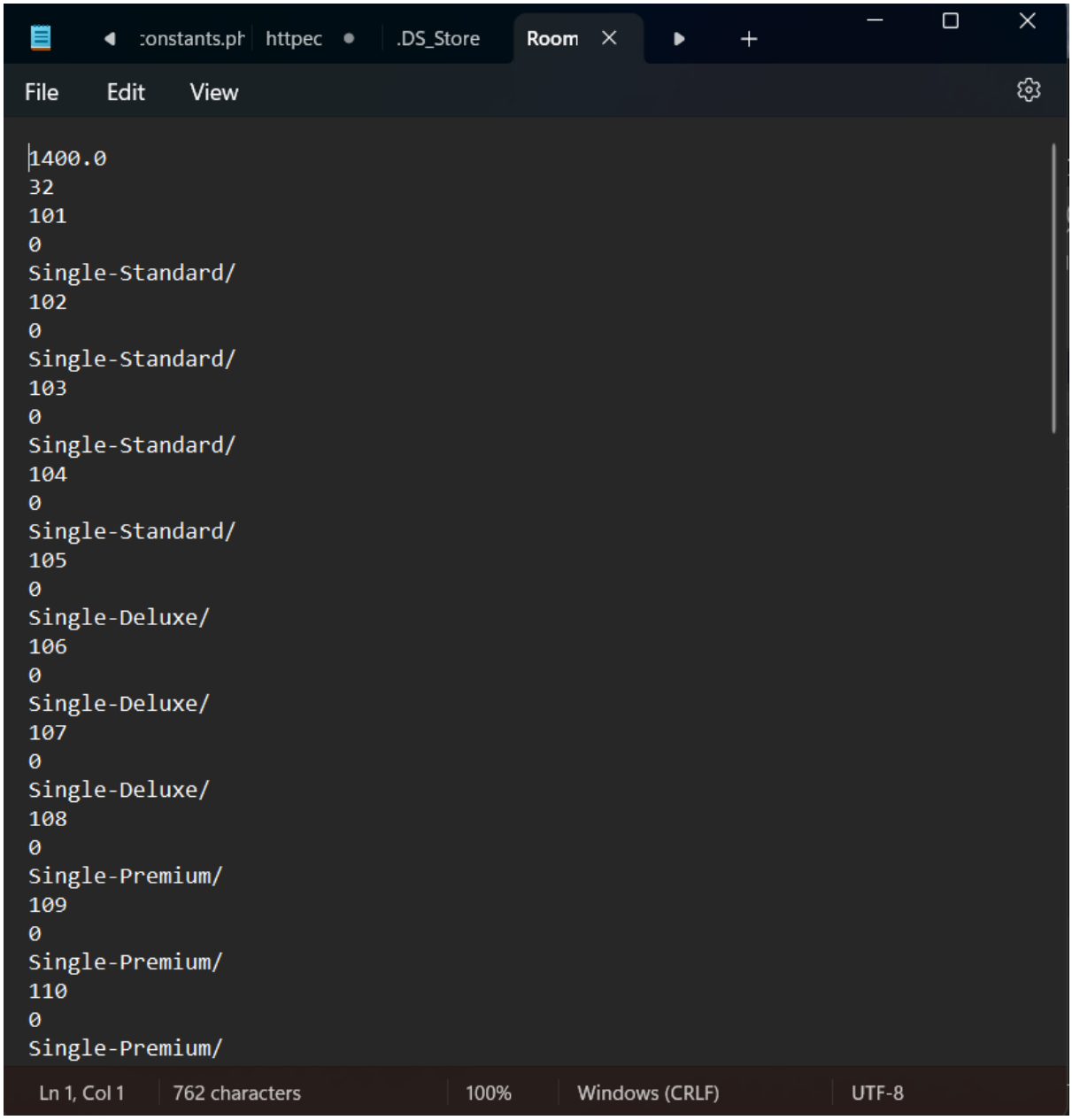
```
Console
<terminated> StageD [Java Application] C:\Program Files\Java\jdk-22\bin\javaw.exe (18 Aug 2024, 11:21:53 – 11:37:15) [p

-MAIN MENU-
-----------
1. Display All Rooms' Details
2. Check Details of a specific room
3. Check-In
4. Check-Out
5. Show Summary
6. Exit
Enter your choice : 6

Program Terminating.....
```

TextFile being Created




:onstants.ph
httpec
.DS_Store
Room
File
Edit
View
1400.0
32
101
0
Single-Standard/
102
0
Single-Standard/
103
0
Single-Standard/
104
0
Single-Standard/
105
0
Single-Deluxe/
106
0
Single-Deluxe/
107
0
Single-Deluxe/
108
0
Single-Premium/
109
0
Single-Premium/
110
0
Single-Premium/
Ln 1, Col 1
762 characters
100%
Windows (CRLF)
UTF-8